

Internship Assignment

Core Java – Inheritance, Method Overriding and Array Processing

Project Title

Bank Account Management System using Inheritance and Arrays

1. Problem Statement

A bank wants to build a **simple account management system** to manage different types of bank accounts.

All bank accounts share some **common operations**, such as:

- Depositing money
- Withdrawing money
- Checking account details
- Calculating interest

However, different account types follow **different business rules**.

For example:

- **Savings Accounts** provide interest on the account balance.
- **Current Accounts** allow an **overdraft facility**, meaning the account holder can withdraw more money than the available balance up to a certain limit.

To avoid code duplication and to design a flexible system, the bank has decided to use **Inheritance in Java**.

The system must demonstrate the **IS-A relationship**, where:

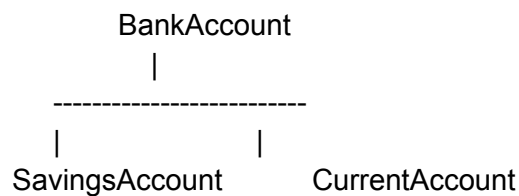
SavingsAccount IS-A BankAccount

CurrentAccount IS-A BankAccount

The system must also manage **multiple accounts**, so all accounts should be stored in an **array of BankAccount references** and processed using a **foreach loop**.

2. System Architecture

The system must follow this class hierarchy:



3. Class Requirements

3.1 Parent Class – BankAccount

Create a class called **BankAccount**.

This class represents a **general bank account**.

Instance Variables

The class should contain the following attributes:

accountNumber

accountHolderName

balance

Constructor

The constructor should initialize all account details.

Example:

```
BankAccount(int accountNumber, String accountHolderName, double balance)
```

Methods

1. deposit(double amount)

Purpose: Add money to the account.

Behavior:

$\text{balance} = \text{balance} + \text{amount}$

Display a confirmation message showing the updated balance.

2. withdraw(double amount)

Purpose: Withdraw money from the account.

Rules:

- If the balance is sufficient → withdrawal should be successful
 - Otherwise → display a message **"Insufficient Balance"**
-

3. displayAccountDetails()

This method should display:

Account Number
Account Holder Name
Current Balance

4. calculateInterest()

This method will be overridden by child classes.

Default behavior:

No interest for generic bank account

3.2 Child Class – SavingsAccount

Create a class **SavingsAccount** that extends **BankAccount**.

Savings accounts provide **interest on the account balance**.

Additional Variable

interestRate

Example:

interestRate = 4%

Method Overriding

Override the method:

calculateInterest()

Formula:

$\text{interest} = \text{balance} * \text{interestRate} / 100$

Add the calculated interest to the balance and display the updated balance.

3.3 Child Class – CurrentAccount

Create a class **CurrentAccount** that extends **BankAccount**.

Current accounts allow an **overdraft facility**.

Additional Variable

overdraftLimit

Example:

overdraftLimit = 5000

Method Overriding

Override the method:

withdraw(double amount)

Rules:

The withdrawal is allowed if:

balance + overdraftLimit >= withdrawal amount

Otherwise display:

Withdrawal denied – overdraft limit exceeded

4. Using Array to Store Accounts

The bank system must handle **multiple accounts**, so the application should store all accounts in an **array of BankAccount references**.

Example concept:

BankAccount[] accounts

This array can store objects of:

SavingsAccount

CurrentAccount

because both classes **inherit from BankAccount**.

5. Main Application

Create a class called:

BankApplication

This class will test the system.

Step 1 – Create an Array

Create an array capable of storing **multiple accounts**.

Example:

```
BankAccount[] accounts = new BankAccount[4];
```

Step 2 – Store Different Account Objects

Store different types of account objects in the array.

Example:

```
accounts[0] = new SavingsAccount(101, "Rahul", 10000, 4);  
accounts[1] = new CurrentAccount(201, "Amit", 5000, 5000);  
accounts[2] = new SavingsAccount(102, "Neha", 8000, 3);  
accounts[3] = new CurrentAccount(202, "Rohit", 6000, 4000);
```

This demonstrates that a **parent class array can store child objects**.

Step 3 – Process Accounts using Foreach Loop

Now iterate through the array using a **foreach loop** and perform operations on every account.

Example operations:

- Display account details
- Deposit money
- Withdraw money
- Calculate interest

Example concept:

```
for(BankAccount acc : accounts)
{
    acc.displayAccountDetails();
    acc.deposit(1000);
    acc.withdraw(2000);
    acc.calculateInterest();
}
```

The **JVM will decide which method to execute at runtime**, depending on the actual object type.

This demonstrates **Runtime Polymorphism**.